

```
    {  
        s = 0x8000;  
        comp = -comp;  
    }  
    return (unsigned short) (s | comp);  
}  
}
```

Related topics

[Texture block compression](#)

BC7 format

Article • 10/20/2022

The BC7 format is a texture compression format used for high-quality compression of RGB and RGBA data.

For info about the block modes of the BC7 format, see [BC7 Format Mode Reference](#).

About BC7/DXGI_FORMAT_BC7

BC7 is specified by the following DXGI_FORMAT enumeration values:

- `DXGI_FORMAT_BC7_TYPELESS`.
- `DXGI_FORMAT_BC7_UNORM`.
- `DXGI_FORMAT_BC7_UNORM_SRGB`.

The BC7 format can be used for [Texture2D](#) (including arrays), `Texture3D`, or `TextureCube` (including arrays) texture resources. Similarly, this format applies to any MIP-map surfaces associated with these resources.

BC7 uses a fixed block size of 16 bytes (128 bits) and a fixed tile size of 4x4 texels. As with previous BC formats, texture images larger than the supported tile size (4x4) are compressed by using multiple blocks. This addressing identity also applies to three-dimensional images and MIP-maps, cubemaps, and texture arrays. All image tiles must be of the same format.

BC7 compresses both three-channel (RGB) and four-channel (RGBA) fixed-point data images. Typically, source data is 8 bits per color component (channel), although the format is capable of encoding source data with higher bits per color component. All image tiles must be of the same format.

The BC7 decoder performs decompression before texture filtering is applied.

BC7 decompression hardware must be bit accurate; that is, the hardware must return results that are identical to the results returned by the decoder described in this document.

BC7 Implementation

A BC7 implementation can specify one of 8 modes, with the mode specified in the least significant bit of the 16 byte (128 bit) block. The mode is encoded by zero or more bits

with a value of 0 followed by a 1.

A BC7 block can contain multiple endpoint pairs. The set of indices that correspond to an endpoint pair may be referred to as a "subset." Also, in some block modes, the endpoint representation is encoded in a form that can be referred to as "RBGP," where the "P" bit represents a shared least significant bit for the color components of the endpoint. For example, if the endpoint representation for the format is "RGB 5.5.5.1," then the endpoint is interpreted as an RGB 6.6.6 value, where the state of the P-bit defines the least significant bit of each component. Similarly, for source data with an alpha channel, if the representation for the format is "RGBAP 5.5.5.5.1," then the endpoint is interpreted as RGBA 6.6.6.6. Depending on the block mode, you can specify the shared least significant bit for either both endpoints of a subset individually (2 P-bits per subset), or shared between endpoints of a subset (1 P-bit per subset).

For BC7 blocks that don't explicitly encode the alpha component, a BC7 block consists of mode bits, partition bits, compressed endpoints, compressed indices, and an optional P-bit. In these blocks the endpoints have an RGB-only representation and the alpha component is decoded as 1.0 for all texels in the source data.

For BC7 blocks that have combined color and alpha components, a block consists of mode bits, compressed endpoints, compressed indices, and optional partition bits and a P-bit. In these blocks, the endpoint colors are expressed in RGBA format, and alpha component values are interpolated alongside the color component values.

For BC7 blocks that have separate color and alpha components, a block consists of mode bits, rotation bits, compressed endpoints, compressed indices, and an optional index selector bit. These blocks have an effective RGB vector [R, G, B] and a scalar alpha channel [A] separately encoded.

The following table lists the components of each block type.

BC7 block contains...	mode bits	rotation bits	index selector bit	partition bits	compressed endpoints	P-bit	compressed indices
color components only	required	N/A	N/A	required	required	optional	required
color + alpha combined	required	N/A	N/A	optional	required	optional	required

BC7 block contains...	mode bits	rotation bits	index selector bit	partition bits	compressed endpoints	P-bit	compressed indices
color and alpha separated	required	required	optional	N/A	required	N/A	required

BC7 defines a palette of colors on an approximate line between two endpoints. The mode value determines the number of interpolating endpoint pairs per block. BC7 stores one palette index per texel.

For each subset of indices that corresponds to a pair of endpoints, the encoder fixes the state of one bit of the compressed index data for that subset. It does so by choosing an endpoint order that allows the index for the designated "fix-up" index to set its most significant bit to 0, and which can then be discarded, saving one bit per subset. For block modes with only a single subset, the fix-up index is always index 0.

Decoding the BC7 Format

The following pseudocode outlines the steps to decompress the pixel at (x,y) given the 16 byte BC7 block.

syntax

```
decompress_bc7(x, y, block)
{
    mode = extract_mode(block);

    //decode partition data from explicit partition bits
    subset_index = 0;
    num_subsets = 1;

    if (mode.type == 0 OR == 1 OR == 2 OR == 3 OR == 7)
    {
        num_subsets = get_num_subsets(mode.type);
        partition_set_id = extract_partition_set_id(mode, block);
        subset_index = get_partition_index(num_subsets, partition_set_id, x,
y);
    }

    //extract raw, compressed endpoint bits
    UINT8 endpoint_array[num_subsets][4] = extract_endpoints(mode, block);

    //decode endpoint color and alpha for each subset
    fully_decode_endpoints(endpoint_array, mode, block);
```

```

//endpoints are now complete.
UINT8 endpoint_start[4] = endpoint_array[2 * subset_index];
UINT8 endpoint_end[4]   = endpoint_array[2 * subset_index + 1];

//Determine the palette index for this pixel
alpha_index      = get_alpha_index(block, mode, x, y);
alpha_bitcount   = get_alpha_bitcount(block, mode);
color_index      = get_color_index(block, mode, x, y);
color_bitcount   = get_color_bitcount(block, mode);

//determine output
UINT8 output[4];
output.rgb = interpolate(endpoint_start.rgb, endpoint_end.rgb,
color_index, color_bitcount);
output.a   = interpolate(endpoint_start.a,   endpoint_end.a,
alpha_index, alpha_bitcount);

if (mode.type == 4 OR == 5)
{
    //Decode the 2 color rotation bits as follows:
    // 00 - Block format is Scalar(A) Vector(RGB) - no swapping
    // 01 - Block format is Scalar(R) Vector(AGB) - swap A and R
    // 10 - Block format is Scalar(G) Vector(RAB) - swap A and G
    // 11 - Block format is Scalar(B) Vector(RGA) - swap A and B
    rotation = extract_rot_bits(mode, block);
    output = swap_channels(output, rotation);
}
}

```

The following pseudocode outlines the steps to fully decode endpoint color and alpha components for each subset given a 16-byte BC7 block.

syntax

```

fully_decode_endpoints(endpoint_array, mode, block)
{
    //first handle modes that have P-bits
    if (mode.type == 0 OR == 1 OR == 3 OR == 6 OR == 7)
    {
        for each endpoint i
        {
            //component-wise left-shift
            endpoint_array[i].rgba = endpoint_array[i].rgba << 1;
        }

        //if P-bit is shared
        if (mode.type == 1)
        {
            pbit_zero = extract_pbit_zero(mode, block);
            pbit_one  = extract_pbit_one(mode, block);
        }
    }
}

```

```

        //rgb component-wise insert pbits
        endpoint_array[0].rgb |= pbit_zero;
        endpoint_array[1].rgb |= pbit_zero;
        endpoint_array[2].rgb |= pbit_one;
        endpoint_array[3].rgb |= pbit_one;
    }
    else //unique P-bit per endpoint
    {
        pbit_array = extract_pbit_array(mode, block);
        for each endpoint i
        {
            endpoint_array[i].rgba |= pbit_array[i];
        }
    }
}

for each endpoint i
{
    // Color_component_precision & alpha_component_precision includes
pbit
    // left shift endpoint components so that their MSB lies in bit 7
    endpoint_array[i].rgb = endpoint_array[i].rgb << (8 -
color_component_precision(mode));
    endpoint_array[i].a = endpoint_array[i].a << (8 -
alpha_component_precision(mode));

    // Replicate each component's MSB into the LSBs revealed by the
left-shift operation above
    endpoint_array[i].rgb = endpoint_array[i].rgb |
(endpoint_array[i].rgb >> color_component_precision(mode));
    endpoint_array[i].a = endpoint_array[i].a | (endpoint_array[i].a >>
alpha_component_precision(mode));
}

//If this mode does not explicitly define the alpha component
//set alpha equal to 1.0
if (mode.type == 0 OR == 1 OR == 2 OR == 3)
{
    for each endpoint i
    {
        endpoint_array[i].a = 255; //i.e. alpha = 1.0f
    }
}
}

```

To generate each interpolated component for each subset, use the following algorithm: let "c" be the component to generate; let "e0" be that component of endpoint 0 of the subset; and let "e1" be that component of endpoint 1 of the subset.

```

UINT16 aWeight2[] = {0, 21, 43, 64};
UINT16 aWeight3[] = {0, 9, 18, 27, 37, 46, 55, 64};
UINT16 aWeight4[] = {0, 4, 9, 13, 17, 21, 26, 30, 34, 38, 43, 47, 51, 55, 60, 64};

UINT8 interpolate(UINT8 e0, UINT8 e1, UINT8 index, UINT8 indexprecision)
{
    if(indexprecision == 2)
        return (UINT8) (((64 - aWeights2[index])*UINT16(e0) +
aWeights2[index]*UINT16(e1) + 32) >> 6);
    else if(indexprecision == 3)
        return (UINT8) (((64 - aWeights3[index])*UINT16(e0) +
aWeights3[index]*UINT16(e1) + 32) >> 6);
    else // indexprecision == 4
        return (UINT8) (((64 - aWeights4[index])*UINT16(e0) +
aWeights4[index]*UINT16(e1) + 32) >> 6);
}

```

The following pseudocode illustrates how to extract indices and bit counts for color and alpha components. Blocks with separate color and alpha also have two sets of index data: one for the vector channel, and one for the scalar channel. For Mode 4, these indices are of differing widths (2 or 3 bits), and there is a one-bit selector which specifies whether the vector or scalar data uses the 3-bit indices. (Extracting the alpha bit count is similar to extracting color bit count but with inverse behavior based on the **idxMode** bit.)

syntax

```

bitcount get_color_bitcount(block, mode)
{
    if (mode.type == 0 OR == 1)
        return 3;

    if (mode.type == 2 OR == 3 OR == 5 OR == 7)
        return 2;

    if (mode.type == 6)
        return 4;

    //The only remaining case is Mode 4 with 1-bit index selector
    idxMode = extract_idxMode(block);
    if (idxMode == 0)
        return 2;
    else
        return 3;
}

```

BC7 format mode reference

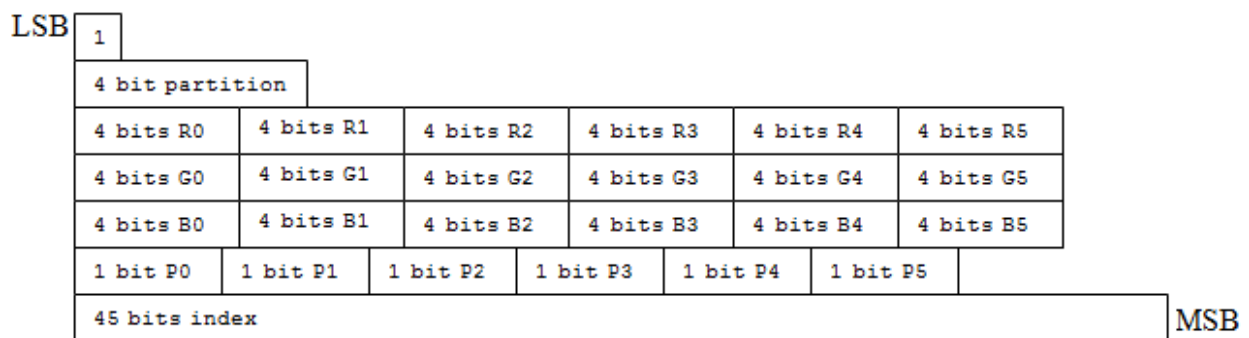
This section contains a list of the 8 block modes and bit allocations for BC7 texture compression format blocks.

The colors for each subset within a block are represented by two explicit endpoint colors and a set of interpolated colors between them. Depending on the block's index precision, each subset can have 4, 8 or 16 possible colors.

Mode 0

BC7 Mode 0 has the following characteristics:

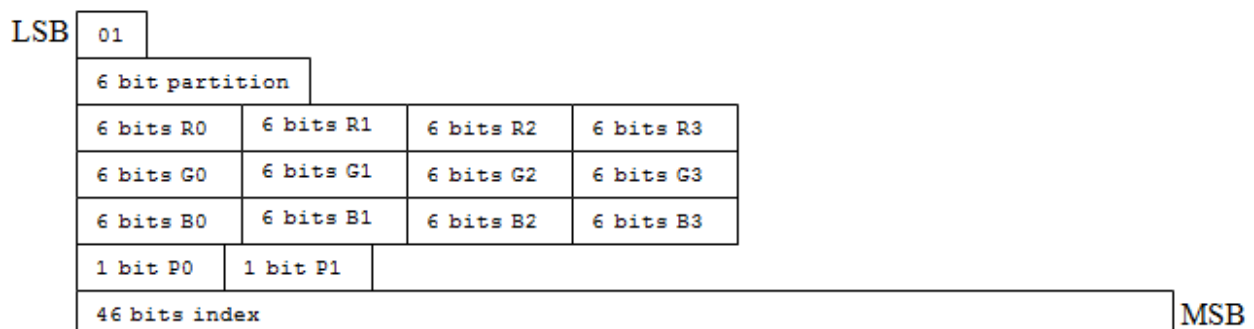
- Color components only (no alpha)
- 3 subsets per block
- RGBP 4.4.4.1 endpoints with a unique P-bit per endpoint
- 3-bit indices
- 16 partitions



Mode 1

BC7 Mode 1 has the following characteristics:

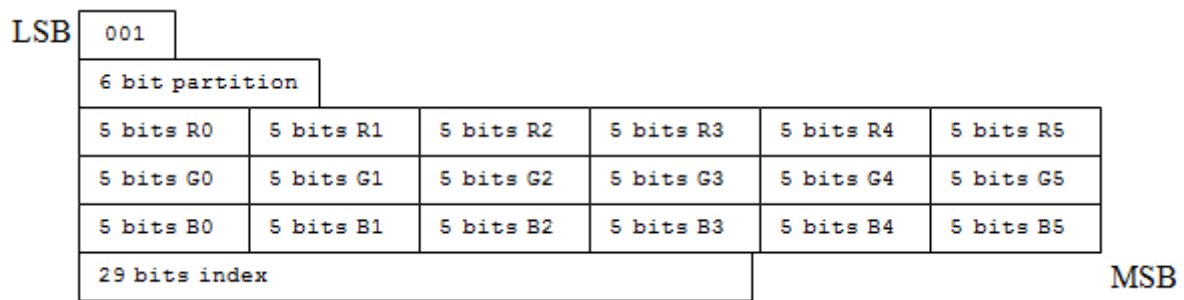
- Color components only (no alpha)
- 2 subsets per block
- RGBP 6.6.6.1 endpoints with a shared P-bit per subset)
- 3-bit indices
- 64 partitions



Mode 2

BC7 Mode 2 has the following characteristics:

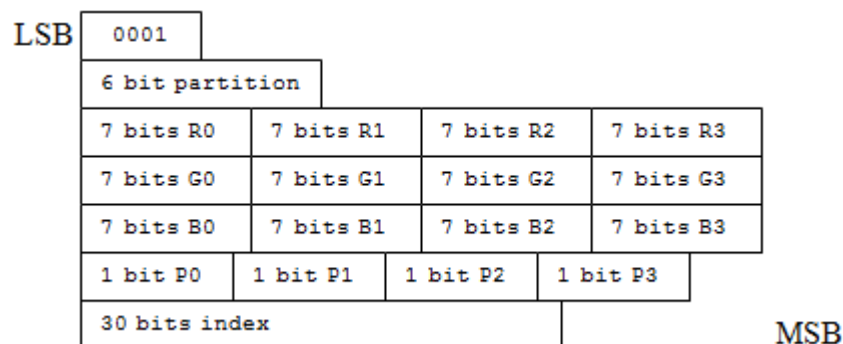
- Color components only (no alpha)
- 3 subsets per block
- RGB 5.5.5 endpoints
- 2-bit indices
- 64 partitions



Mode 3

BC7 Mode 3 has the following characteristics:

- Color components only (no alpha)
- 2 subsets per block
- RGBP 7.7.7.1 endpoints with a unique P-bit per subset)
- 2-bit indices
- 64 partitions

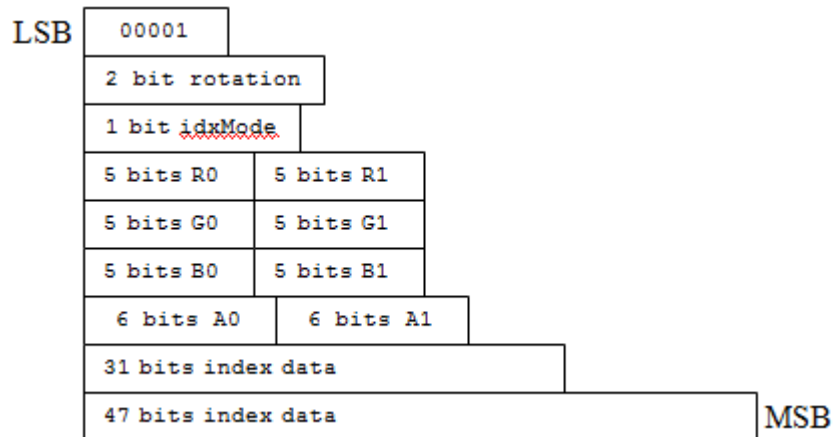


Mode 4

BC7 Mode 4 has the following characteristics:

- Color components with separate alpha component
- 1 subset per block
- RGB 5.5.5 color endpoints

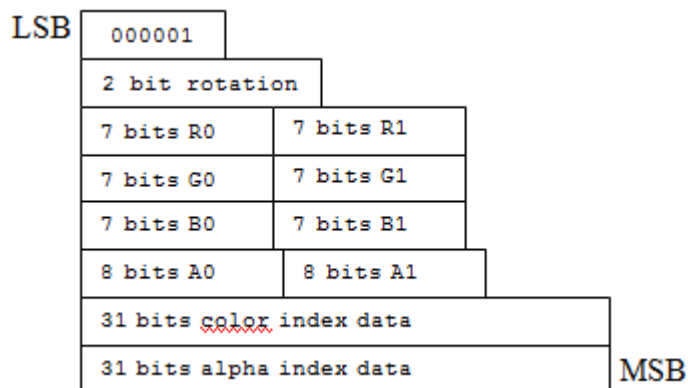
- 6-bit alpha endpoints
- 16 x 2-bit indices
- 16 x 3-bit indices
- 2-bit component rotation
- 1-bit index selector (whether the 2- or 3-bit indices are used)



Mode 5

BC7 Mode 5 has the following characteristics:

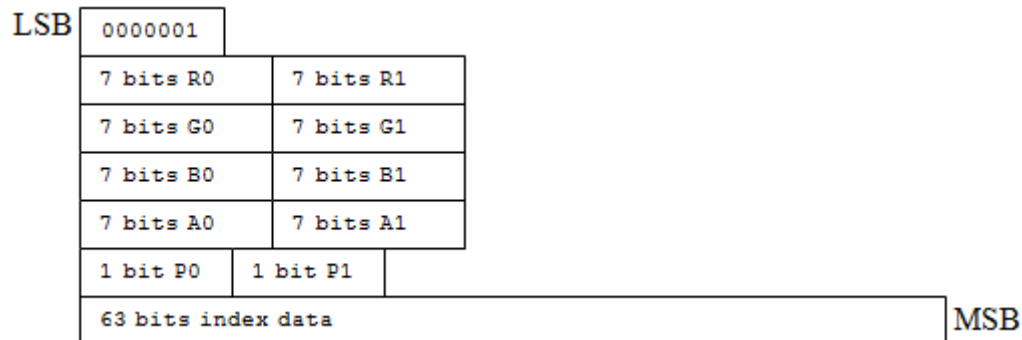
- Color components with separate alpha component
- 1 subset per block
- RGB 7.7.7 color endpoints
- 6-bit alpha endpoints
- 16 x 2-bit color indices
- 16 x 2-bit alpha indices
- 2-bit component rotation



Mode 6

BC7 Mode 6 has the following characteristics:

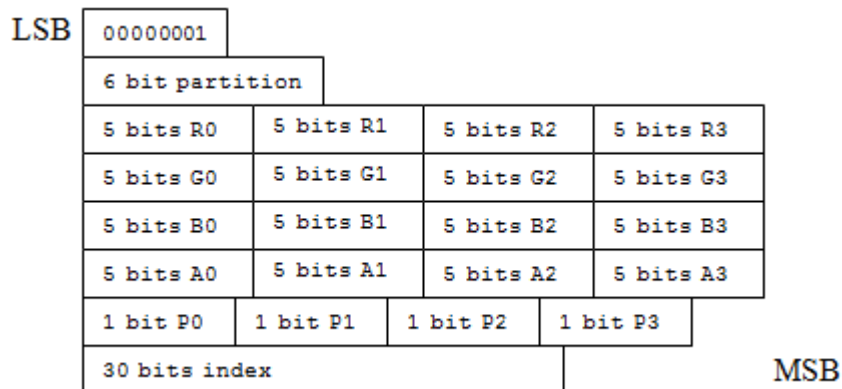
- Combined color and alpha components
- One subset per block
- RGBAP 7.7.7.7.1 color (and alpha) endpoints (unique P-bit per endpoint)
- 16 x 4-bit indices



Mode 7

BC7 Mode 7 has the following characteristics:

- Combined color and alpha components
- 2 subsets per block
- RGBAP 5.5.5.5.1 color (and alpha) endpoints (unique P-bit per endpoint)
- 2-bit indices
- 64 partitions



Remarks

Mode 8 (the least significant byte is set to 0x00) is reserved. Don't use it in your encoder. If you pass this mode to the hardware, a block initialized to all zeroes is returned.

In BC7, you can encode the alpha component in one of the following ways:

- Block types without explicit alpha component encoding. In these blocks, the color endpoints have an RGB-only encoding, with the alpha component decoded to 1.0 for all texels.

- Block types with combined color and alpha components. In these blocks, the endpoint color values are specified in the RGBA format, and the alpha component values are interpolated along with the color values.
- Block types with separated color and alpha components. In these blocks the color and alpha values are specified separately, each with their own set of indices. As a result, they have an effective vector and a scalar channel separately encoded, where the vector commonly specifies the color channels [R, G, B] and the scalar specifies the alpha channel [A]. To support this approach, a separate 2-bit field is provided in the encoding, which permits the specification of the separate channel encoding as a scalar value. As a result, the block can have one of the following four different representations of this alpha encoding (as indicated by the 2-bit field):
 - RGB|A: alpha channel separate
 - AGB|R: "red" color channel separate
 - RAB|G: "green" color channel separate
 - RGA|B: "blue" color channel separate

The decoder reorders the channel order back to RGBA after decoding, so the internal block format is invisible to the developer. Blocks with separate color and alpha components also have two sets of index data: one for the vectored set of channels, and one for the scalar channel. (In the case of Mode 4, these indices are of differing widths [2 or 3 bits]. Mode 4 also contains a 1-bit selector that specifies whether the vector or the scalar channel uses the 3-bit indices.)

Related topics

[Texture block compression](#)

Fundamentals of DirectX programming

Article • 10/20/2022

This section provides information about the basic blocks of DirectX programming.

[2D graphics for DirectX games](#) topic explains the use of 2D bitmap graphics and effects, and how to use them in your game using a combination of elements from both Direct2D and Direct3D libraries.

[Direct3D graphics learning guide](#) topic provides an overview of the concepts that Direct3D uses.

[Basic 3D graphics for DirectX games](#) topic explains the fundamental concepts of 3D graphics using a five-part tutorial that introduces the Direct3D concept and API. This tutorial shows you how to initialize Direct3D interfaces using Windows Runtime, apply per-vertex shader operations, set up the geometry, rasterize the scene, and cull hidden surfaces.

[Load resources in your DirectX game](#) topic guides you through the basic steps for loading meshes (models), textures (bitmaps), and compiled shader objects from local storage or other data streams for use in your UWP game.

Topic	Description
2D graphics for DirectX games	Create 2D graphics using DirectX.
Direct3D graphics learning guide	Understand Direct3D graphics concepts.
Basic 3D graphics for DirectX games	Create basic 3D DirectX graphics.
Load resources in your DirectX game	Load meshes in your DirectX game.

2D graphics for DirectX games

Article • 10/20/2022

We discuss the use of 2D bitmap graphics and effects, and how to use them in your game.

2D graphics are a subset of 3D graphics that deal with 2D primitives or bitmaps. More generally, they don't use a z-coordinate in the way a 3D game might, since the game play is usually confined to the x-y plane. They sometimes use 3D graphics techniques to create their visual components, and they are generally simpler to develop. If you are new to gaming, a 2D game is a great place to start, and 2D graphics development can be a good place for you to get a handle on DirectX.

You can develop 2D gaming graphics in DirectX using either Direct2D or Direct3D, or some combination. Many of the more useful classes for 2D game development are in Direct3D, such as the [Sprite](#) class. Direct2D is a set of APIs that primarily target user interfaces and apps that require support for drawing primitives (such as circles, lines, and flat polygon shapes). With that in mind, it still provides a powerful and performant set of classes and methods for creating game graphics as well, especially when creating game overlays, interfaces, and heads-up displays (HUDs) -- or for creating a variety of 2D games, from simple to reasonably detailed. The most effective approach when creating 2D games, though, is to use elements from both libraries, and that's the way we will approach 2D graphics development in this topic.

Concepts at a glance

Before the advent of modern 3D graphics and the hardware that supports it, games were primarily 2D, and many of their graphics techniques involved moving blocks of memory around -- usually arrays of color data that would be translated or transformed to pixels on the screen in a 1:1 fashion.

In DirectX, 2D graphics are part of the 3D pipeline. There is a much greater variety of screen resolutions and graphics hardware available, and your 2D graphics engine must be able to support them without a significant change in fidelity.

Here are a few of the basic concepts you should be familiar with when starting 2D graphics development.

- Pixels and raster coordinates. A pixel is a single point on a raster display, and has its own (x, y) coordinate pair indicating its location on the display. (The term "pixel" is often used interchangeably between the physical pixels that comprise the

display and the addressable memory elements used to hold the color and alpha values of the pixels before they are sent to the display.) The raster is treated by APIs as a rectangular grid of pixel elements, which often has a 1:1 correspondence with the physical pixel grid of a display. Raster coordinate systems start from the upper left, with the pixel at (0, 0) in the upper leftmost corner of the grid.

- Bitmap graphics (sometimes called raster graphics) are graphic elements represented as a rectangular grid of pixel values. Sprites -- computed pixel arrays managed independent of the raster -- are one type of bitmap graphic, commonly used for the active characters or background-independent animated objects in a game. The various frames of animation for a sprite are represented as collections of bitmaps called "sheets" or "batches." Backgrounds are larger bitmap objects that are the same resolution or greater than that of the screen raster, and often serve as the backdrop(s) for a game's playfield.
- Vector graphics are graphics that use geometric primitives, such as points, lines, circles, and polygons to define 2D objects. They are represented not as arrays of pixels, but as the mathematical equations that define them in a 2D space. They do not necessarily have a 1:1 correspondence with the pixel grid of the display, and must be transformed from the coordinate system that you rendered them in into the raster coordinate system of the display.
- Translation is when you take a point or vertex and calculate its new location in the same coordinate system.
- Scaling is when you enlarge or shrink an object by a specified scale factor. With a vector image, you shrink and enlarge its component vertices; with a bitmap, you enlarge the pixel elements or diminish them. With bitmap images, you lose pixel data when the image shrinks, and you enlarge the individual pixels when the image is scaled closer. For the latter, you can use pixel color interpolation operations, like bilinear filtering, to smooth out the harsh color boundaries between the enlarged pixels.
- Rotation is when you rotate an object about a specified axis or axes. With a vector image, the vertices of the geometry are multiplied against a rotation matrix to obtain the rotated vertex; with a bitmap image, different algorithms can be employed, each with a lesser or greater degree of fidelity in the results. As with scaling and translation, there are APIs specifically for rotation operations.
- Transformation is when you take one point or vertex in one coordinate system and calculate its corresponding point or vertex in another coordinate system. This includes translation, scaling, and rotation, as well as other coordinate calculation operations.
- Clipping is when you remove portions of bitmaps or geometry that are not within the viewable area of the display, or are hidden by objects with higher view priority.

- The frame buffer is an area in memory -- often in the memory of the graphics hardware itself -- that contains the final raster map that you will draw to the screen. The swap chain is a collection of buffers, where you draw in a back buffer and, when the image is ready, you "swap" it to the front and display it.

Design considerations

2D graphics development is a great way to get accustomed to developing with Direct3D, and will allow you to spend more time on other critical aspects of game development: audio, controls, and the game mechanics.

Always draw to a back buffer. Drawing directly to your frame buffer means that your image will be displayed when the signal for display is received (usually every 1/60th of second), even if your drawing operation hasn't completed!

Design your graphics engine to support a good selection of resolutions, from 1024x600 to 1920x1080 (or higher). Your audience will thank you if you support their LCD monitor's native resolution, especially with 2D graphics.

Great artwork will be your greatest asset, when it comes to visuals. While your bitmap graphics may lack the punch of 3D photorealistic visuals using the latest shader model features, great high-resolution artwork can often convey as much or more style and personality -- and with far less of a performance penalty.

Reference

- [Direct2D overview](#)
- [Direct2D quickstart](#)
- [Direct2D and Direct3D interoperability overview](#)

Basic 3D graphics for DirectX games

Article • 10/20/2022

We show how to use DirectX programming to implement the fundamental concepts of 3D graphics.

Objective: Learn to program a 3D graphics app.

Prerequisites

We assume that you are familiar with C++. You also need basic experience with graphics programming concepts.

Total time to complete: 30 minutes.

Where to go from here

Here, we talk about how to develop 3D graphics with DirectX and C++\C#. This five-part tutorial introduces you to the [Direct3D](#) API and the concepts and code that are also used in many of the other DirectX samples. These parts build upon each other, from configuring DirectX for your UWP C++ app to texturing primitives and adding effects.

Note This tutorial uses a right-handed coordinate system with column vectors. Many DirectX samples and apps use a left-handed coordinate system with row vectors. For a more complete graphics math solution and one that supports a left-handed coordinate system with row vectors, consider using [DirectXMath](#). For more info, see [Using DirectXMath with Direct3D](#).

We show you how to:

- Initialize [Direct3D](#) interfaces by using the Windows Runtime
- Apply per-vertex shader operations
- Set up the geometry
- Rasterize the scene (flattening the 3D scene to a 2D projection)
- Cull the hidden surfaces

Note

Next, we create a Direct3D device, swap chain, and render-target view, and present the rendered image to the display.

[Quickstart: setting up DirectX resources and displaying an image](#)

Related topics

- [Direct3D 11 Graphics](#)
- [DXGI](#)
- [HLSL](#)

Set up DirectX resources and display an image

Article • 10/20/2022

Here, we show you how to create a Direct3D device, swap chain, and render-target view, and how to present the rendered image to the display.

Objective: To set up DirectX resources in a C++ Universal Windows Platform (UWP) app and to display a solid color.

Prerequisites

We assume that you are familiar with C++. You also need basic experience with graphics programming concepts.

Time to complete: 20 minutes.

Instructions

1. Declaring Direct3D interface variables with ComPtr

We declare Direct3D interface variables with the ComPtr [smart pointer](#) template from the Windows Runtime C++ Template Library (WRL), so we can manage the lifetime of those variables in an exception safe manner. We can then use those variables to access the [ComPtr class](#) and its members. For example:

C++

```
ComPtr<ID3D11RenderTargetView> m_renderTargetView;  
m_d3dDeviceContext->OMSetRenderTargets(  
    1,  
    m_renderTargetView.GetAddressOf(),  
    nullptr // Use no depth stencil.  
);
```

If you declare [ID3D11RenderTargetView](#) with ComPtr, you can then use ComPtr's [GetAddressOf](#) method to get the address of the pointer to [ID3D11RenderTargetView](#) (**ID3D11RenderTargetView) to pass to [ID3D11DeviceContext::OMSetRenderTargets](#). [OMSetRenderTargets](#) binds the render target to the [output-merger stage](#) to specify the render target as the output target.

After the sample app is started, it initializes and loads, and is then ready to run.

2. Creating the Direct3D device

To use the Direct3D API to render a scene, we must first create a Direct3D device that represents the display adapter. To create the Direct3D device, we call the `D3D11CreateDevice` function. We specify levels 9.1 through 11.1 in the array of `D3D_FEATURE_LEVEL` values. Direct3D walks the array in order and returns the highest supported feature level. So, to get the highest feature level available, we list the `D3D_FEATURE_LEVEL` array entries from highest to lowest. We pass the `D3D11_CREATE_DEVICE_BGRA_SUPPORT` flag to the *Flags* parameter to make Direct3D resources interoperate with Direct2D. If we use the debug build, we also pass the `D3D11_CREATE_DEVICE_DEBUG` flag. For more info about debugging apps, see [Using the debug layer to debug apps](#).

We obtain the Direct3D 11.1 device (`ID3D11Device1`) and device context (`ID3D11DeviceContext1`) by querying the Direct3D 11 device and device context that are returned from `D3D11CreateDevice`.

C++

```
// First, create the Direct3D device.

// This flag is required in order to enable compatibility with
Direct2D.
UINT creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

#ifdef _DEBUG
    // If the project is in a debug build, enable debugging via SDK
    Layers with this flag.
    creationFlags |= D3D11_CREATE_DEVICE_DEBUG;
#endif

// This array defines the ordering of feature levels that D3D should
attempt to create.
D3D_FEATURE_LEVEL featureLevels[] =
{
    D3D_FEATURE_LEVEL_11_1,
    D3D_FEATURE_LEVEL_11_0,
    D3D_FEATURE_LEVEL_10_1,
    D3D_FEATURE_LEVEL_10_0,
    D3D_FEATURE_LEVEL_9_3,
    D3D_FEATURE_LEVEL_9_1
};

ComPtr<ID3D11Device> d3dDevice;
ComPtr<ID3D11DeviceContext> d3dDeviceContext;
DX::ThrowIfFailed(
```

```

        D3D11CreateDevice(
            nullptr,                // Specify nullptr to use the
default adapter.
            D3D_DRIVER_TYPE_HARDWARE,
            nullptr,                // leave as nullptr if hardware
is used
            creationFlags,          // optionally set debug and
Direct2D compatibility flags
            featureLevels,
            ARRAYSIZE(featureLevels),
            D3D11_SDK_VERSION,     // always set this to
D3D11_SDK_VERSION
            &d3dDevice,
            nullptr,
            &d3dDeviceContext
        )
    );

    // Retrieve the Direct3D 11.1 interfaces.
    DX::ThrowIfFailed(
        d3dDevice.As(&m_d3dDevice)
    );

    DX::ThrowIfFailed(
        d3dDeviceContext.As(&m_d3dDeviceContext)
    );

```

3. Creating the swap chain

Next, we create a swap chain that the device uses for rendering and display. We declare and initialize a [DXGI_SWAP_CHAIN_DESC1](#) structure to describe the swap chain. Then, we set up the swap chain as flip-model (that is, a swap chain that has the [DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL](#) value set in the **SwapEffect** member) and set the **Format** member to [DXGI_FORMAT_B8G8R8A8_UNORM](#). We set the **Count** member of the [DXGI_SAMPLE_DESC](#) structure that the **SampleDesc** member specifies to 1 and the **Quality** member of [DXGI_SAMPLE_DESC](#) to zero because flip-model doesn't support multiple sample antialiasing (MSAA). We set the **BufferCount** member to 2 so the swap chain can use a front buffer to present to the display device and a back buffer that serves as the render target.

We obtain the underlying DXGI device by querying the Direct3D 11.1 device. To minimize power consumption, which is important to do on battery-powered devices such as laptops and tablets, we call the [IDXGIDevice1::SetMaximumFrameLatency](#) method with 1 as the maximum number of back buffer frames that DXGI can queue. This ensures that the app is rendered only after the vertical blank.

To finally create the swap chain, we need to get the parent factory from the DXGI device. We call `IDXGIDevice::GetAdapter` to get the adapter for the device, and then call `IDXGIObject::GetParent` on the adapter to get the parent factory (`IDXGIFactory2`). To create the swap chain, we call `IDXGIFactory2::CreateSwapChainForCoreWindow` with the swap-chain descriptor and the app's core window.

C++

```
// If the swap chain does not exist, create it.
DXGI_SWAP_CHAIN_DESC1 swapChainDesc = {0};

swapChainDesc.Stereo = false;
swapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
swapChainDesc.Scaling = DXGI_SCALING_NONE;
swapChainDesc.Flags = 0;

// Use automatic sizing.
swapChainDesc.Width = 0;
swapChainDesc.Height = 0;

// This is the most common swap chain format.
swapChainDesc.Format = DXGI_FORMAT_B8G8R8A8_UNORM;

// Don't use multi-sampling.
swapChainDesc.SampleDesc.Count = 1;
swapChainDesc.SampleDesc.Quality = 0;

// Use two buffers to enable the flip effect.
swapChainDesc.BufferCount = 2;

// We recommend using this swap effect for all applications.
swapChainDesc.SwapEffect = DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL;

// Once the swap chain description is configured, it must be
// created on the same adapter as the existing D3D Device.

// First, retrieve the underlying DXGI Device from the D3D
Device.
ComPtr<IDXGIDevice2> dxgiDevice;
DX::ThrowIfFailed(
    m_d3dDevice.As(&dxgiDevice)
);

// Ensure that DXGI does not queue more than one frame at a
time. This both reduces
// latency and ensures that the application will only render
after each VSync, minimizing
// power consumption.
DX::ThrowIfFailed(
    dxgiDevice->SetMaximumFrameLatency(1)
);
```

```

// Next, get the parent factory from the DXGI Device.
ComPtr<IDXGIAdapter> dxgiAdapter;
DX::ThrowIfFailed(
    dxgiDevice->GetAdapter(&dxgiAdapter)
);

ComPtr<IDXGIFactory2> dxgiFactory;
DX::ThrowIfFailed(
    dxgiAdapter->GetParent(IID_PPV_ARGS(&dxgiFactory))
);

// Finally, create the swap chain.
CoreWindow^ window = m_window.Get();
DX::ThrowIfFailed(
    dxgiFactory->CreateSwapChainForCoreWindow(
        m_d3dDevice.Get(),
        reinterpret_cast<IUnknown*>(window),
        &swapChainDesc,
        nullptr, // Allow on all displays.
        &m_swapChain
    )
);

```

4. Creating the render-target view

To render graphics to the window, we need to create a render-target view. We call [IDXGISwapChain::GetBuffer](#) to get the swap chain's back buffer to use when we create the render-target view. We specify the back buffer as a 2D texture ([ID3D11Texture2D](#)). To create the render-target view, we call [ID3D11Device::CreateRenderTargetView](#) with the swap chain's back buffer. We must specify to draw to the entire core window by specifying the view port ([D3D11_VIEWPORT](#)) as the full size of the swap chain's back buffer. We use the view port in a call to [ID3D11DeviceContext::RSSetViewports](#) to bind the view port to the [rasterizer stage](#) of the pipeline. The rasterizer stage converts vector information into a raster image. In this case, we don't require a conversion because we are just displaying a solid color.

C++

```

// Once the swap chain is created, create a render target view.
This will
// allow Direct3D to render graphics to the window.

ComPtr<ID3D11Texture2D> backBuffer;
DX::ThrowIfFailed(
    m_swapChain->GetBuffer(0, IID_PPV_ARGS(&backBuffer))
);

DX::ThrowIfFailed(
    m_d3dDevice->CreateRenderTargetView(

```

```

        backBuffer.Get(),
        nullptr,
        &m_renderTargetView
    );

    // After the render target view is created, specify that the
    viewport,
    // which describes what portion of the window to draw to, should
    cover
    // the entire window.

    D3D11_TEXTURE2D_DESC backBufferDesc = {0};
    backBuffer->GetDesc(&backBufferDesc);

    D3D11_VIEWPORT viewport;
    viewport.TopLeftX = 0.0f;
    viewport.TopLeftY = 0.0f;
    viewport.Width = static_cast<float>(backBufferDesc.Width);
    viewport.Height = static_cast<float>(backBufferDesc.Height);
    viewport.MinDepth = D3D11_MIN_DEPTH;
    viewport.MaxDepth = D3D11_MAX_DEPTH;

    m_d3dDeviceContext->RSSetViewports(1, &viewport);

```

5. Presenting the rendered image

We enter an endless loop to continually render and display the scene.

In this loop, we call:

1. [ID3D11DeviceContext::OMSetRenderTargets](#) to specify the render target as the output target.
2. [ID3D11DeviceContext::ClearRenderTargetView](#) to clear the render target to a solid color.
3. [IDXGISwapChain::Present](#) to present the rendered image to the window.

Because we previously set the maximum frame latency to 1, Windows generally slows down the render loop to the screen refresh rate, typically around 60 Hz. Windows slows down the render loop by making the app sleep when the app calls [Present](#). Windows makes the app sleep until the screen is refreshed.

C++

```

    // Enter the render loop. Note that UWP apps should never exit.
    while (true)
    {
        // Process events incoming to the window.

```



```

        m_window->Dispatcher-
        >ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

        // Specify the render target we created as the output target.
        m_d3dDeviceContext->OMSetRenderTargets(
            1,
            m_renderTargetView.GetAddressOf(),
            nullptr // Use no depth stencil.
        );

        // Clear the render target to a solid color.
        const float clearColor[4] = { 0.071f, 0.04f, 0.561f, 1.0f };
        m_d3dDeviceContext->ClearRenderTargetView(
            m_renderTargetView.Get(),
            clearColor
        );

        // Present the rendered image to the window. Because the
        maximum frame latency is set to 1,
        // the render loop will generally be throttled to the screen
        refresh rate, typically around
        // 60 Hz, by sleeping the application on Present until the
        screen is refreshed.
        DX::ThrowIfFailed(
            m_swapChain->Present(1, 0)
        );
    }

```

6. Resizing the app window and the swap chain's buffer

If the size of the app window changes, the app must resize the swap chain's buffers, recreate the render-target view, and then present the resized rendered image. To resize the swap chain's buffers, we call [IDXGISwapChain::ResizeBuffers](#). In this call, we leave the number of buffers and the format of the buffers unchanged (the *BufferCount* parameter to two and the *NewFormat* parameter to [DXGI_FORMAT_B8G8R8A8_UNORM](#)). We make the size of the swap chain's back buffer the same size as the resized window. After we resize the swap chain's buffers, we create the new render target and present the new rendered image similarly to when we initialized the app.

C++

```

        // If the swap chain already exists, resize it.
        DX::ThrowIfFailed(
            m_swapChain->ResizeBuffers(
                2,
                0,
                0,
                DXGI_FORMAT_B8G8R8A8_UNORM,

```

```
    0  
    )  
);
```

Summary and next steps

We created a Direct3D device, swap chain, and render-target view, and presented the rendered image to the display.

Next, we also draw a triangle on the display.

[Creating shaders and drawing primitives](#)

Create shaders and drawing primitives

Article • 10/20/2022

Here, we show you how to use HLSL source files to compile and create shaders that you can then use to draw primitives on the display.

We create and draw a yellow triangle by using vertex and pixel shaders. After we create the Direct3D device, the swap chain, and the render-target view, we read data from binary shader object files on the disk.

Objective: To create shaders and to draw primitives.

Prerequisites

We assume that you are familiar with C++. You also need basic experience with graphics programming concepts.

We also assume that you went through [Quickstart: setting up DirectX resources and displaying an image](#).

Time to complete: 20 minutes.

Instructions

1. Compiling HLSL source files

Microsoft Visual Studio uses the [fxc.exe](#) HLSL code compiler to compile the .hlsl source files (SimpleVertexShader.hlsl and SimplePixelShader.hlsl) into .cso binary shader object files (SimpleVertexShader.cso and SimplePixelShader.cso). For more info about the HLSL code compiler, see Effect-Compiler Tool. For more info about compiling shader code, see [Compiling Shaders](#).

Here is the code in SimpleVertexShader.hlsl:

hlsl

```
struct VertexShaderInput
{
    DirectX::XMFLOAT2 pos : POSITION;
};

struct PixelShaderInput
{

```

```

float4 pos : SV_POSITION;
};

PixelShaderInput SimpleVertexShader(VertexShaderInput input)
{
    PixelShaderInput vertexShaderOutput;

    // For this lesson, set the vertex depth value to 0.5, so it is
    // guaranteed to be drawn.
    vertexShaderOutput.pos = float4(input.pos, 0.5f, 1.0f);

    return vertexShaderOutput;
}

```

Here is the code in SimplePixelShader.hlsl:

```

hls

struct PixelShaderInput
{
    float4 pos : SV_POSITION;
};

float4 SimplePixelShader(PixelShaderInput input) : SV_TARGET
{
    // Draw the entire triangle yellow.
    return float4(1.0f, 1.0f, 0.0f, 1.0f);
}

```

2. Reading data from disk

We use the `DX::ReadDataAsync` function from `DirectXHelper.h` in the DirectX 11 App (Universal Windows) template to asynchronously read data from a file on the disk.

3. Creating vertex and pixel shaders

We read data from the `SimpleVertexShader.cso` file and assign the data to the `vertexShaderBytecode` byte array. We call `ID3D11Device::CreateVertexShader` with the byte array to create the vertex shader (`ID3D11VertexShader`). We set the vertex depth value to 0.5 in the `SimpleVertexShader.hlsl` source to guarantee that our triangle is drawn. We populate an array of `D3D11_INPUT_ELEMENT_DESC` structures to describe the layout of the vertex shader code and then call `ID3D11Device::CreateInputLayout` to create the layout. The array has one layout element that defines the vertex position. We read data from the `SimplePixelShader.cso` file and assign the data to the `pixelShaderBytecode` byte array. We call `ID3D11Device::CreatePixelShader` with the byte array to create the pixel shader (`ID3D11PixelShader`). We set the pixel value to (1,1,1,1) in

the SimplePixelShader.hlsl source to make our triangle yellow. You can change the color by changing this value.

We create vertex and index buffers that define a simple triangle. To do this, we first define the triangle, next describe the vertex and index buffers ([D3D11_BUFFER_DESC](#) and [D3D11_SUBRESOURCE_DATA](#)) using the triangle definition, and finally call [ID3D11Device::CreateBuffer](#) once for each buffer.

C++

```
auto loadVSTask = DX::ReadDataAsync(L"SimpleVertexShader.cso");
auto loadPSTask = DX::ReadDataAsync(L"SimplePixelShader.cso");

// Load the raw vertex shader bytecode from disk and create a vertex
shader with it.
auto createVSTask = loadVSTask.then([this](const std::vector<byte>&
vertexShaderBytecode) {

    ComPtr<ID3D11VertexShader> vertexShader;
    DX::ThrowIfFailed(
        m_d3dDevice->CreateVertexShader(
            vertexShaderBytecode->Data,
            vertexShaderBytecode->Length,
            nullptr,
            &vertexShader
        )
    );

    // Create an input layout that matches the layout defined in the
vertex shader code.
    // For this lesson, this is simply a DirectX::XMFLOAT2 vector
defining the vertex position.
    const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
    {
        { "POSITION", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 0,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
    };

    ComPtr<ID3D11InputLayout> inputLayout;
    DX::ThrowIfFailed(
        m_d3dDevice->CreateInputLayout(
            basicVertexLayoutDesc,
            ARRAYSIZE(basicVertexLayoutDesc),
            vertexShaderBytecode->Data,
            vertexShaderBytecode->Length,
            &inputLayout
        )
    );
});

// Load the raw pixel shader bytecode from disk and create a pixel
```

shader with it.

```
    auto createPSTask = loadPSTask.then([this](const std::vector<byte>&
pixelShaderBytecode) {
        ComPtr<ID3D11PixelShader> pixelShader;
        DX::ThrowIfFailed(
            m_d3dDevice->CreatePixelShader(
                pixelShaderBytecode->Data,
                pixelShaderBytecode->Length,
                nullptr,
                &pixelShader
            )
        );
    });
```

// Create vertex and index buffers that define a simple triangle.

```
    auto createTriangleTask = (createPSTask && createVSTask).then([this]
() {
```

```
        DirectX::XMFLOAT2 triangleVertices[] =
        {
            float2(-0.5f, -0.5f),
            float2( 0.0f,  0.5f),
            float2( 0.5f, -0.5f),
        };
```

```
        unsigned short triangleIndices[] =
        {
            0, 1, 2,
        };
```

```
        D3D11_BUFFER_DESC vertexBufferDesc = {0};
        vertexBufferDesc.ByteWidth = sizeof(float2) *
ARRAYSIZE(triangleVertices);
        vertexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
        vertexBufferDesc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
        vertexBufferDesc.CPUAccessFlags = 0;
        vertexBufferDesc.MiscFlags = 0;
        vertexBufferDesc.StructureByteStride = 0;
```

```
        D3D11_SUBRESOURCE_DATA vertexBufferData;
        vertexBufferData.pSysMem = triangleVertices;
        vertexBufferData.SysMemPitch = 0;
        vertexBufferData.SysMemSlicePitch = 0;
```

```
        ComPtr<ID3D11Buffer> vertexBuffer;
        DX::ThrowIfFailed(
            m_d3dDevice->CreateBuffer(
                &vertexBufferDesc,
                &vertexBufferData,
                &vertexBuffer
            )
        );
```

```
        D3D11_BUFFER_DESC indexBufferDesc;
        indexBufferDesc.ByteWidth = sizeof(unsigned short) *
```

```

ARRAYSIZE(triangleIndices);
    indexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
    indexBufferDesc.BindFlags = D3D11_BIND_INDEX_BUFFER;
    indexBufferDesc.CPUAccessFlags = 0;
    indexBufferDesc.MiscFlags = 0;
    indexBufferDesc.StructureByteStride = 0;

    D3D11_SUBRESOURCE_DATA indexBufferData;
    indexBufferData.pSysMem = triangleIndices;
    indexBufferData.SysMemPitch = 0;
    indexBufferData.SysMemSlicePitch = 0;

    ComPtr<ID3D11Buffer> indexBuffer;
    DX::ThrowIfFailed(
        m_d3dDevice->CreateBuffer(
            &indexBufferDesc,
            &indexBufferData,
            &indexBuffer
        )
    );
});

```

We use the vertex and pixel shaders, the vertex shader layout, and the vertex and index buffers to draw a yellow triangle.

4. Drawing the triangle and presenting the rendered image

We enter an endless loop to continually render and display the scene. We call [ID3D11DeviceContext::OMSetRenderTargets](#) to specify the render target as the output target. We call [ID3D11DeviceContext::ClearRenderTargetView](#) with { 0.071f, 0.04f, 0.561f, 1.0f } to clear the render target to a solid blue color.

In the endless loop, we draw a yellow triangle on the blue surface.

To draw a yellow triangle

1. First, we call [ID3D11DeviceContext::IASetInputLayout](#) to describe how vertex buffer data is streamed into the input-assembler stage.
2. Next, we call [ID3D11DeviceContext::IASetVertexBuffers](#) and [ID3D11DeviceContext::IASetIndexBuffer](#) to bind the vertex and index buffers to the input-assembler stage.
3. Next, we call [ID3D11DeviceContext::IASetPrimitiveTopology](#) with the [D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP](#) value to specify for the input-assembler stage to interpret the vertex data as a triangle strip.

4. Next, we call `ID3D11DeviceContext::VSSetShader` to initialize the vertex shader stage with the vertex shader code and `ID3D11DeviceContext::PSSetShader` to initialize the pixel shader stage with the pixel shader code.
5. Finally, we call `ID3D11DeviceContext::DrawIndexed` to draw the triangle and submit it to the rendering pipeline.

We call `IDXGISwapChain::Present` to present the rendered image to the window.

C++

```
// Specify the render target we created as the output target.
m_d3dDeviceContext->OMSetRenderTargets(
    1,
    m_renderTargetView.GetAddressOf(),
    nullptr // Use no depth stencil.
);

// Clear the render target to a solid color.
const float clearColor[4] = { 0.071f, 0.04f, 0.561f, 1.0f };
m_d3dDeviceContext->ClearRenderTargetView(
    m_renderTargetView.Get(),
    clearColor
);

m_d3dDeviceContext->IASetInputLayout(inputLayout.Get());

// Set the vertex and index buffers, and specify the way they
define geometry.
UINT stride = sizeof(float2);
UINT offset = 0;
m_d3dDeviceContext->IASetVertexBuffers(
    0,
    1,
    vertexBuffer.GetAddressOf(),
    &stride,
    &offset
);

m_d3dDeviceContext->IASetIndexBuffer(
    indexBuffer.Get(),
    DXGI_FORMAT_R16_UINT,
    0
);

m_d3dDeviceContext-
>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);

// Set the vertex and pixel shader stage state.
m_d3dDeviceContext->VSSetShader(
    vertexShader.Get(),
    nullptr,
    0
```



```

    );

    m_d3dDeviceContext->PSSetShader(
        pixelShader.Get(),
        nullptr,
        0
    );

    // Draw the cube.
    m_d3dDeviceContext->DrawIndexed(
        ARRAYSIZE(triangleIndices),
        0,
        0
    );

    // Present the rendered image to the window. Because the
    maximum frame latency is set to 1,
    // the render loop will generally be throttled to the screen
    refresh rate, typically around
    // 60 Hz, by sleeping the application on Present until the
    screen is refreshed.
    DX::ThrowIfFailed(
        m_swapChain->Present(1, 0)
    );

```

Summary and next steps

We created and drew a yellow triangle by using vertex and pixel shaders.

Next, we create an orbiting 3D cube and apply lighting effects to it.

[Using depth and effects on primitives](#)

Use depth and effects on primitives

Article • 10/20/2022

Here, we show you how to use depth, perspective, color, and other effects on primitives.

Objective: To create a 3D object and apply basic vertex lighting and coloring to it.

Prerequisites

We assume that you are familiar with C++. You also need basic experience with graphics programming concepts.

We also assume that you went through [Quickstart: setting up DirectX resources and displaying an image](#) and [Creating shaders and drawing primitives](#).

Time to complete: 20 minutes.

Instructions

1. Defining cube variables

First, we need to define the **SimpleCubeVertex** and **ConstantBuffer** structures for the cube. These structures specify the vertex positions and colors for the cube and how the cube will be viewed. We declare **ID3D11DepthStencilView** and **ID3D11Buffer** with **ComPtr** and declare an instance of **ConstantBuffer**.

C++

```
struct SimpleCubeVertex
{
    DirectX::XMFLOAT3 pos;    // Position
    DirectX::XMFLOAT3 color;  // Color
};

struct ConstantBuffer
{
    DirectX::XMFLOAT4X4 model;
    DirectX::XMFLOAT4X4 view;
    DirectX::XMFLOAT4X4 projection;
};

// This class defines the application as a whole.
ref class Direct3DTutorialFrameworkView : public IFrameworkView
{
```

```
private:
    Platform::Agile<CoreWindow> m_window;
    ComPtr<IDXGISwapChain1> m_swapChain;
    ComPtr<ID3D11Device1> m_d3dDevice;
    ComPtr<ID3D11DeviceContext1> m_d3dDeviceContext;
    ComPtr<ID3D11RenderTargetView> m_renderTargetView;
    ComPtr<ID3D11DepthStencilView> m_depthStencilView;
    ComPtr<ID3D11Buffer> m_constantBuffer;
    ConstantBuffer m_constantBufferData;
```

2. Creating a depth stencil view

In addition to creating the render-target view, we also create a depth-stencil view. The depth-stencil view enables Direct3D to efficiently render objects closer to the camera in front of objects further from the camera. Before we can create a view to a depth-stencil buffer, we must create the depth-stencil buffer. We populate a [D3D11_TEXTURE2D_DESC](#) to describe the depth-stencil buffer and then call [ID3D11Device::CreateTexture2D](#) to create the depth-stencil buffer. To create the depth-stencil view, we populate a [D3D11_DEPTH_STENCIL_VIEW_DESC](#) to describe the depth-stencil view and pass the depth-stencil view description and the depth-stencil buffer to [ID3D11Device::CreateDepthStencilView](#).

C++

```
// Once the render target view is created, create a depth stencil
view. This
// allows Direct3D to efficiently render objects closer to the
camera in front
// of objects further from the camera.

D3D11_TEXTURE2D_DESC backBufferDesc = {0};
backBuffer->GetDesc(&backBufferDesc);

D3D11_TEXTURE2D_DESC depthStencilDesc;
depthStencilDesc.Width = backBufferDesc.Width;
depthStencilDesc.Height = backBufferDesc.Height;
depthStencilDesc.MipLevels = 1;
depthStencilDesc.ArraySize = 1;
depthStencilDesc.Format = DXGI_FORMAT_D24_UNORM_S8_UINT;
depthStencilDesc.SampleDesc.Count = 1;
depthStencilDesc.SampleDesc.Quality = 0;
depthStencilDesc.Usage = D3D11_USAGE_DEFAULT;
depthStencilDesc.BindFlags = D3D11_BIND_DEPTH_STENCIL;
depthStencilDesc.CPUAccessFlags = 0;
depthStencilDesc.MiscFlags = 0;
ComPtr<ID3D11Texture2D> depthStencil;
DX::ThrowIfFailed(
    m_d3dDevice->CreateTexture2D(
        &depthStencilDesc,
```

```

        nullptr,
        &depthStencil
    )
);

D3D11_DEPTH_STENCIL_VIEW_DESC depthStencilViewDesc;
depthStencilViewDesc.Format = depthStencilDesc.Format;
depthStencilViewDesc.ViewDimension = D3D11_DSV_DIMENSION_TEXTURE2D;
depthStencilViewDesc.Flags = 0;
depthStencilViewDesc.Texture2D.MipSlice = 0;
DX::ThrowIfFailed(
    m_d3dDevice->CreateDepthStencilView(
        depthStencil.Get(),
        &depthStencilViewDesc,
        &m_depthStencilView
    )
);

```

3. Updating perspective with the window

We update the perspective projection parameters for the constant buffer depending on the window dimensions. We fix the parameters to a 70-degree field of view with a depth range of 0.01 to 100.

C++

```

    // Finally, update the constant buffer perspective projection
    parameters
    // to account for the size of the application window. In this
    sample,
    // the parameters are fixed to a 70-degree field of view, with a
    depth
    // range of 0.01 to 100. For a generalized camera class, see Lesson
    5.

    float xScale = 1.42814801f;
    float yScale = 1.42814801f;
    if (backBufferDesc.Width > backBufferDesc.Height)
    {
        xScale = yScale *
            static_cast<float>(backBufferDesc.Height) /
            static_cast<float>(backBufferDesc.Width);
    }
    else
    {
        yScale = xScale *
            static_cast<float>(backBufferDesc.Width) /
            static_cast<float>(backBufferDesc.Height);
    }

    m_constantBufferData.projection = DirectX::XMFOAT4X4(

```

```
xScale, 0.0f, 0.0f, 0.0f,
0.0f, yScale, 0.0f, 0.0f,
0.0f, 0.0f, -1.0f, -0.01f,
0.0f, 0.0f, -1.0f, 0.0f
);
```

4. Creating vertex and pixel shaders with color elements

In this app, we create more complex vertex and pixel shaders than what we described in the previous tutorial, [Creating shaders and drawing primitives](#). The app's vertex shader transforms each vertex position into projection space and passes the vertex color through to the pixel shader.

The app's array of `D3D11_INPUT_ELEMENT_DESC` structures that describe the layout of the vertex shader code has two layout elements: one element defines the vertex position and the other element defines the color.

We create vertex, index, and constant buffers to define an orbiting cube.

To define an orbiting cube

1. First, we define the cube. We assign each vertex a color in addition to a position. This allows the pixel shader to color each face differently so the face can be distinguished.
2. Next, we describe the vertex and index buffers (`D3D11_BUFFER_DESC` and `D3D11_SUBRESOURCE_DATA`) using the cube definition. We call `ID3D11Device::CreateBuffer` once for each buffer.
3. Next, we create a constant buffer (`D3D11_BUFFER_DESC`) for passing model, view, and projection matrices to the vertex shader. We can later use the constant buffer to rotate the cube and apply a perspective projection to it. We call `ID3D11Device::CreateBuffer` to create the constant buffer.
4. Next, we specify the view transform that corresponds to a camera position of $X = 0$, $Y = 1$, $Z = 2$.
5. Finally, we declare a *degree* variable that we will use to animate the cube by rotating it every frame.

C++

```
auto loadVSTask = DX::ReadDataAsync(L"SimpleVertexShader.cso");
auto loadPSTask = DX::ReadDataAsync(L"SimplePixelShader.cso");

auto createVSTask = loadVSTask.then([this](const std::vector<byte>&
vertexShaderBytecode) {
```

```

ComPtr<ID3D11VertexShader> vertexShader;
DX::ThrowIfFailed(
    m_d3dDevice->CreateVertexShader(
        vertexShaderBytecode->Data,
        vertexShaderBytecode->Length,
        nullptr,
        &vertexShader
    )
);

// Create an input layout that matches the layout defined in the
vertex shader code.
// For this lesson, this is simply a DirectX::XMFLOAT3 vector
defining the vertex position, and
// a DirectX::XMFLOAT3 vector defining the vertex color.
const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "COLOR", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
};

ComPtr<ID3D11InputLayout> inputLayout;
DX::ThrowIfFailed(
    m_d3dDevice->CreateInputLayout(
        basicVertexLayoutDesc,
        ARRAYSIZE(basicVertexLayoutDesc),
        vertexShaderBytecode->Data,
        vertexShaderBytecode->Length,
        &inputLayout
    )
);

});

// Load the raw pixel shader bytecode from disk and create a pixel
shader with it.
auto createPSTask = loadPSTask.then([this](const std::vector<byte>&
pixelShaderBytecode) {
    ComPtr<ID3D11PixelShader> pixelShader;
    DX::ThrowIfFailed(
        m_d3dDevice->CreatePixelShader(
            pixelShaderBytecode->Data,
            pixelShaderBytecode->Length,
            nullptr,
            &pixelShader
        )
    );
});

// Create vertex and index buffers that define a simple unit cube.
auto createCubeTask = (createPSTask && createVSTask).then([this] ()
{

```

```

        // In the array below, which will be used to initialize the cube
vertex buffers,
        // each vertex is assigned a color in addition to a position.
This will allow
        // the pixel shader to color each face differently, enabling them
to be distinguished.
        SimpleCubeVertex cubeVertices[] =
        {
            { float3(-0.5f, 0.5f, -0.5f), float3(0.0f, 1.0f, 0.0f) }, //
+Y (top face)
            { float3( 0.5f, 0.5f, -0.5f), float3(1.0f, 1.0f, 0.0f) },
            { float3( 0.5f, 0.5f,  0.5f), float3(1.0f, 1.0f, 1.0f) },
            { float3(-0.5f, 0.5f,  0.5f), float3(0.0f, 1.0f, 1.0f) },

            { float3(-0.5f, -0.5f,  0.5f), float3(0.0f, 0.0f, 1.0f) }, //
-Y (bottom face)
            { float3( 0.5f, -0.5f,  0.5f), float3(1.0f, 0.0f, 1.0f) },
            { float3( 0.5f, -0.5f, -0.5f), float3(1.0f, 0.0f, 0.0f) },
            { float3(-0.5f, -0.5f, -0.5f), float3(0.0f, 0.0f, 0.0f) },
        };

        unsigned short cubeIndices[] =
        {
            0, 1, 2,
            0, 2, 3,

            4, 5, 6,
            4, 6, 7,

            3, 2, 5,
            3, 5, 4,

            2, 1, 6,
            2, 6, 5,

            1, 7, 6,
            1, 0, 7,

            0, 3, 4,
            0, 4, 7
        };

        D3D11_BUFFER_DESC vertexBufferDesc = {0};
        vertexBufferDesc.ByteWidth = sizeof(SimpleCubeVertex) *
ARRAYSIZE(cubeVertices);
        vertexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
        vertexBufferDesc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
        vertexBufferDesc.CPUAccessFlags = 0;
        vertexBufferDesc.MiscFlags = 0;
        vertexBufferDesc.StructureByteStride = 0;

        D3D11_SUBRESOURCE_DATA vertexBufferData;
        vertexBufferData.pSysMem = cubeVertices;
        vertexBufferData.SysMemPitch = 0;

```

```

vertexBufferData.SysMemSlicePitch = 0;

ComPtr<ID3D11Buffer> vertexBuffer;
DX::ThrowIfFailed(
    m_d3dDevice->CreateBuffer(
        &vertexBufferDesc,
        &vertexBufferData,
        &vertexBuffer
    )
);

D3D11_BUFFER_DESC indexBufferDesc;
indexBufferDesc.ByteWidth = sizeof(unsigned short) *
ARRAYSIZE(cubeIndices);
indexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
indexBufferDesc.BindFlags = D3D11_BIND_INDEX_BUFFER;
indexBufferDesc.CPUAccessFlags = 0;
indexBufferDesc.MiscFlags = 0;
indexBufferDesc.StructureByteStride = 0;

D3D11_SUBRESOURCE_DATA indexBufferData;
indexBufferData.pSysMem = cubeIndices;
indexBufferData.SysMemPitch = 0;
indexBufferData.SysMemSlicePitch = 0;

ComPtr<ID3D11Buffer> indexBuffer;
DX::ThrowIfFailed(
    m_d3dDevice->CreateBuffer(
        &indexBufferDesc,
        &indexBufferData,
        &indexBuffer
    )
);

// Create a constant buffer for passing model, view, and
projection matrices
// to the vertex shader. This will allow us to rotate the cube
and apply
// a perspective projection to it.

D3D11_BUFFER_DESC constantBufferDesc = {0};
constantBufferDesc.ByteWidth = sizeof(m_constantBufferData);
constantBufferDesc.Usage = D3D11_USAGE_DEFAULT;
constantBufferDesc.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
constantBufferDesc.CPUAccessFlags = 0;
constantBufferDesc.MiscFlags = 0;
constantBufferDesc.StructureByteStride = 0;
DX::ThrowIfFailed(
    m_d3dDevice->CreateBuffer(
        &constantBufferDesc,
        nullptr,
        &m_constantBuffer
    )
);

```



```

        // Specify the view transform corresponding to a camera position
of
        // X = 0, Y = 1, Z = 2. For a generalized camera class, see
Lesson 5.

        m_constantBufferData.view = DirectX::XMFLOAT4X4(
            -1.00000000f, 0.00000000f, 0.00000000f, 0.00000000f,
            0.00000000f, 0.89442718f, 0.44721359f, 0.00000000f,
            0.00000000f, 0.44721359f, -0.89442718f, -2.23606800f,
            0.00000000f, 0.00000000f, 0.00000000f, 1.00000000f
        );

    });

    // This value will be used to animate the cube by rotating it every
frame.
    float degree = 0.0f;

```

5. Rotating and drawing the cube and presenting the rendered image

We enter an endless loop to continually render and display the scene. We call the `rotationY` inline function (`BasicMath.h`) with a rotation amount to set values that will rotate the cube's model matrix around the Y axis. We then call `ID3D11DeviceContext::UpdateSubresource` to update the constant buffer and rotate the cube model. We call `ID3D11DeviceContext::OMSetRenderTargets` to specify the render target as the output target. In this `OMSetRenderTargets` call, we pass the depth-stencil view. We call `ID3D11DeviceContext::ClearRenderTargetView` to clear the render target to a solid blue color and call `ID3D11DeviceContext::ClearDepthStencilView` to clear the depth buffer.

In the endless loop, we also draw the cube on the blue surface.

To draw the cube

1. First, we call `ID3D11DeviceContext::IASetInputLayout` to describe how vertex buffer data is streamed into the input-assembler stage.
2. Next, we call `ID3D11DeviceContext::IASetVertexBuffers` and `ID3D11DeviceContext::IASetIndexBuffer` to bind the vertex and index buffers to the input-assembler stage.
3. Next, we call `ID3D11DeviceContext::IASetPrimitiveTopology` with the `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP` value to specify for the input-assembler stage to interpret the vertex data as a triangle strip.

4. Next, we call `ID3D11DeviceContext::VSSetShader` to initialize the vertex shader stage with the vertex shader code and `ID3D11DeviceContext::PSSetShader` to initialize the pixel shader stage with the pixel shader code.
5. Next, we call `ID3D11DeviceContext::VSSetConstantBuffers` to set the constant buffer that is used by the vertex shader pipeline stage.
6. Finally, we call `ID3D11DeviceContext::DrawIndexed` to draw the cube and submit it to the rendering pipeline.

We call `IDXGISwapChain::Present` to present the rendered image to the window.

C++

```
// Update the constant buffer to rotate the cube model.
m_constantBufferData.model = XMMatrixRotationY(-degree);
degree += 1.0f;

m_d3dDeviceContext->UpdateSubresource(
    m_constantBuffer.Get(),
    0,
    nullptr,
    &m_constantBufferData,
    0,
    0
);

// Specify the render target and depth stencil we created as the
output target.
m_d3dDeviceContext->OMSetRenderTargets(
    1,
    m_renderTargetView.GetAddressOf(),
    m_depthStencilView.Get()
);

// Clear the render target to a solid color, and reset the depth
stencil.
const float clearColor[4] = { 0.071f, 0.04f, 0.561f, 1.0f };
m_d3dDeviceContext->ClearRenderTargetView(
    m_renderTargetView.Get(),
    clearColor
);

m_d3dDeviceContext->ClearDepthStencilView(
    m_depthStencilView.Get(),
    D3D11_CLEAR_DEPTH,
    1.0f,
    0
);

m_d3dDeviceContext->IASetInputLayout(inputLayout.Get());

// Set the vertex and index buffers, and specify the way they
define geometry.
```

```

    UINT stride = sizeof(SimpleCubeVertex);
    UINT offset = 0;
    m_d3dDeviceContext->IASetVertexBuffers(
        0,
        1,
        vertexBuffer.GetAddressOf(),
        &stride,
        &offset
    );

    m_d3dDeviceContext->IASetIndexBuffer(
        indexBuffer.Get(),
        DXGI_FORMAT_R16_UINT,
        0
    );

    m_d3dDeviceContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);

    // Set the vertex and pixel shader stage state.
    m_d3dDeviceContext->VSSetShader(
        vertexShader.Get(),
        nullptr,
        0
    );

    m_d3dDeviceContext->VSSetConstantBuffers(
        0,
        1,
        m_constantBuffer.GetAddressOf()
    );

    m_d3dDeviceContext->PSSetShader(
        pixelShader.Get(),
        nullptr,
        0
    );

    // Draw the cube.
    m_d3dDeviceContext->DrawIndexed(
        ARRAYSIZE(cubeIndices),
        0,
        0
    );

    // Present the rendered image to the window. Because the
    maximum frame latency is set to 1,
    // the render loop will generally be throttled to the screen
    refresh rate, typically around
    // 60 Hz, by sleeping the application on Present until the
    screen is refreshed.
    DX::ThrowIfFailed(
        m_swapChain->Present(1, 0)
    );

```

Summary and next steps

We used depth, perspective, color, and other effects on primitives.

Next, we apply textures to primitives.

[Applying textures to primitives](#)

Apply textures to primitives

Article • 10/20/2022

Here, we load raw texture data and apply that data to a 3D primitive by using the cube that we created in [Using depth and effects on primitives](#). We also introduce a simple dot-product lighting model, where the cube surfaces are lighter or darker based on their distance and angle relative to a light source.

Objective: To apply textures to primitives.

Prerequisites

To get the most out of this topic, you'll need to be familiar with C++. You'll also need basic experience with graphics programming concepts. And, ideally, you'll need to have already followed along with [Quickstart: setting up DirectX resources and displaying an image](#), [Creating shaders and drawing primitives](#), and [Using depth and effects on primitives](#).

Time to complete: 20 minutes.

Instructions

1. Defining variables for a textured cube

First, we need to define the **BasicVertex** and **ConstantBuffer** structures for the textured cube. These structures specify the vertex positions, orientations, and textures for the cube and how the cube will be viewed. Otherwise, we declare variables similarly to the previous tutorial, [Using depth and effects on primitives](#).

C++/CX

```
struct BasicVertex
{
    DirectX::XMFLLOAT3 pos;    // Position
    DirectX::XMFLLOAT3 norm;   // Surface normal vector
    DirectX::XMFLLOAT2 tex;    // Texture coordinate
};

struct ConstantBuffer
{
    DirectX::XMFLLOAT4X4 model;
    DirectX::XMFLLOAT4X4 view;
    DirectX::XMFLLOAT4X4 projection;
```

```
};

// This class defines the application as a whole.
ref class Direct3DTutorialFrameworkView : public IFrameworkView
{
private:
    Platform::Agile<CoreWindow> m_window;
    ComPtr<IDXGISwapChain1> m_swapChain;
    ComPtr<ID3D11Device1> m_d3dDevice;
    ComPtr<ID3D11DeviceContext1> m_d3dDeviceContext;
    ComPtr<ID3D11RenderTargetView> m_renderTargetView;
    ComPtr<ID3D11DepthStencilView> m_depthStencilView;
    ComPtr<ID3D11Buffer> m_constantBuffer;
    ConstantBuffer m_constantBufferData;
};
```

2. Creating vertex and pixel shaders with surface and texture elements

Here, we create more complex vertex and pixel shaders than in the previous tutorial, [Using depth and effects on primitives](#). This app's vertex shader transforms each vertex position into projection space and passes the vertex texture coordinate through to the pixel shader.

The app's array of [D3D11_INPUT_ELEMENT_DESC](#) structures that describe the layout of the vertex shader code has three layout elements: one element defines the vertex position, another element defines the surface normal vector (the direction that the surface normally faces), and the third element defines the texture coordinates.

We create vertex, index, and constant buffers that define an orbiting textured cube.

To define an orbiting textured cube

1. First, we define the cube. Each vertex is assigned a position, a surface normal vector, and texture coordinates. We use multiple vertices for each corner to allow different normal vectors and texture coordinates to be defined for each face.
2. Next, we describe the vertex and index buffers ([D3D11_BUFFER_DESC](#) and [D3D11_SUBRESOURCE_DATA](#)) using the cube definition. We call [ID3D11Device::CreateBuffer](#) once for each buffer.
3. Next, we create a constant buffer ([D3D11_BUFFER_DESC](#)) for passing model, view, and projection matrices to the vertex shader. We can later use the constant buffer to rotate the cube and apply a perspective projection to it. We call [ID3D11Device::CreateBuffer](#) to create the constant buffer.
4. Finally, we specify the view transform that corresponds to a camera position of $X = 0$, $Y = 1$, $Z = 2$.

```

auto loadVSTask = DX::ReadDataAsync(L"SimpleVertexShader.cso");
auto loadPSTask = DX::ReadDataAsync(L"SimplePixelShader.cso");

auto createVSTask = loadVSTask.then([this](const std::vector<byte>&
vertexShaderBytecode)
{
    ComPtr<ID3D11VertexShader> vertexShader;
    DX::ThrowIfFailed(
        m_d3dDevice->CreateVertexShader(
            vertexShaderBytecode->Data,
            vertexShaderBytecode->Length,
            nullptr,
            &vertexShader
        )
    );

    // Create an input layout that matches the layout defined in the vertex
    // shader code.
    // These correspond to the elements of the BasicVertex struct defined
    // above.
    const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
    {
        { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
        D3D11_INPUT_PER_VERTEX_DATA, 0 },
        { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
        D3D11_INPUT_PER_VERTEX_DATA, 0 },
        { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
        D3D11_INPUT_PER_VERTEX_DATA, 0 },
    };

    ComPtr<ID3D11InputLayout> inputLayout;
    DX::ThrowIfFailed(
        m_d3dDevice->CreateInputLayout(
            basicVertexLayoutDesc,
            ARRAYSIZE(basicVertexLayoutDesc),
            vertexShaderBytecode->Data,
            vertexShaderBytecode->Length,
            &inputLayout
        )
    );
});

// Load the raw pixel shader bytecode from disk and create a pixel shader
// with it.
auto createPSTask = loadPSTask.then([this](const std::vector<byte>&
pixelShaderBytecode)
{
    ComPtr<ID3D11PixelShader> pixelShader;
    DX::ThrowIfFailed(
        m_d3dDevice->CreatePixelShader(
            pixelShaderBytecode->Data,
            pixelShaderBytecode->Length,

```

```

        nullptr,
        &pixelShader
    )
    );
});

// Create vertex and index buffers that define a simple unit cube.
auto createCubeTask = (createPSTask && createVSTask).then([this]()
{
    // In the array below, which will be used to initialize the cube vertex
    // buffers,
    // multiple vertices are used for each corner to allow different normal
    // vectors and
    // texture coordinates to be defined for each face.
    BasicVertex cubeVertices[] =
    {
        { DirectX::XMFLOAT3(-0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
1.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 0.0f) }, // +Y (top face)
        { DirectX::XMFLOAT3(0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
1.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
        { DirectX::XMFLOAT3(0.5f, 0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
1.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 1.0f) },
        { DirectX::XMFLOAT3(-0.5f, 0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
1.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 1.0f) },

        { DirectX::XMFLOAT3(-0.5f, -0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
-1.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 0.0f) }, // -Y (bottom face)
        { DirectX::XMFLOAT3(0.5f, -0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
-1.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
        { DirectX::XMFLOAT3(0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
-1.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 1.0f) },
        { DirectX::XMFLOAT3(-0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
-1.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 1.0f) },

        { DirectX::XMFLOAT3(0.5f, 0.5f, 0.5f), DirectX::XMFLOAT3(1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 0.0f) }, // +X (right face)
        { DirectX::XMFLOAT3(0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
        { DirectX::XMFLOAT3(0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 1.0f) },
        { DirectX::XMFLOAT3(0.5f, -0.5f, 0.5f), DirectX::XMFLOAT3(1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 1.0f) },

        { DirectX::XMFLOAT3(-0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(-1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 0.0f) }, // -X (left face)
        { DirectX::XMFLOAT3(-0.5f, 0.5f, 0.5f), DirectX::XMFLOAT3(-1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
        { DirectX::XMFLOAT3(-0.5f, -0.5f, 0.5f), DirectX::XMFLOAT3(-1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(1.0f, 1.0f) },
        { DirectX::XMFLOAT3(-0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(-1.0f,
0.0f, 0.0f), DirectX::XMFLOAT2(0.0f, 1.0f) },

        { DirectX::XMFLOAT3(-0.5f, 0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, 1.0f), DirectX::XMFLOAT2(0.0f, 0.0f) }, // +Z (front face)
        { DirectX::XMFLOAT3(0.5f, 0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, 1.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
    }
});

```



```

0.0f, 1.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
    { DirectX::XMFLOAT3(0.5f, -0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, 1.0f), DirectX::XMFLOAT2(1.0f, 1.0f) },
    { DirectX::XMFLOAT3(-0.5f, -0.5f, 0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, 1.0f), DirectX::XMFLOAT2(0.0f, 1.0f) },

    { DirectX::XMFLOAT3(0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, -1.0f), DirectX::XMFLOAT2(0.0f, 0.0f) }, // -Z (back face)
    { DirectX::XMFLOAT3(-0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, -1.0f), DirectX::XMFLOAT2(1.0f, 0.0f) },
    { DirectX::XMFLOAT3(-0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, -1.0f), DirectX::XMFLOAT2(1.0f, 1.0f) },
    { DirectX::XMFLOAT3(0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(0.0f,
0.0f, -1.0f), DirectX::XMFLOAT2(0.0f, 1.0f) },
};

unsigned short cubeIndices[] =
{
    0, 1, 2,
    0, 2, 3,

    4, 5, 6,
    4, 6, 7,

    8, 9, 10,
    8, 10, 11,

    12, 13, 14,
    12, 14, 15,

    16, 17, 18,
    16, 18, 19,

    20, 21, 22,
    20, 22, 23
};

D3D11_BUFFER_DESC vertexBufferDesc = { 0 };
vertexBufferDesc.ByteWidth = sizeof(BasicVertex) *
ARRAYSIZE(cubeVertices);
vertexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
vertexBufferDesc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
vertexBufferDesc.CPUAccessFlags = 0;
vertexBufferDesc.MiscFlags = 0;
vertexBufferDesc.StructureByteStride = 0;

D3D11_SUBRESOURCE_DATA vertexBufferData;
vertexBufferData.pSysMem = cubeVertices;
vertexBufferData.SysMemPitch = 0;
vertexBufferData.SysMemSlicePitch = 0;

ComPtr<ID3D11Buffer> vertexBuffer;
DX::ThrowIfFailed(
    m_d3dDevice->CreateBuffer(
        &vertexBufferDesc,

```

```

        &vertexBufferData,
        &vertexBuffer
    )
};

D3D11_BUFFER_DESC indexBufferDesc;
indexBufferDesc.ByteWidth = sizeof(unsigned short) *
ARRAYSIZE(cubeIndices);
indexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
indexBufferDesc.BindFlags = D3D11_BIND_INDEX_BUFFER;
indexBufferDesc.CPUAccessFlags = 0;
indexBufferDesc.MiscFlags = 0;
indexBufferDesc.StructureByteStride = 0;

D3D11_SUBRESOURCE_DATA indexBufferData;
indexBufferData.pSysMem = cubeIndices;
indexBufferData.SysMemPitch = 0;
indexBufferData.SysMemSlicePitch = 0;

ComPtr<ID3D11Buffer> indexBuffer;
DX::ThrowIfFailed(
    m_d3dDevice->CreateBuffer(
        &indexBufferDesc,
        &indexBufferData,
        &indexBuffer
    )
);

// Create a constant buffer for passing model, view, and projection
matrices
// to the vertex shader. This will allow us to rotate the cube and
apply
// a perspective projection to it.

D3D11_BUFFER_DESC constantBufferDesc = { 0 };
constantBufferDesc.ByteWidth = sizeof(m_constantBufferData);
constantBufferDesc.Usage = D3D11_USAGE_DEFAULT;
constantBufferDesc.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
constantBufferDesc.CPUAccessFlags = 0;
constantBufferDesc.MiscFlags = 0;
constantBufferDesc.StructureByteStride = 0;
DX::ThrowIfFailed(
    m_d3dDevice->CreateBuffer(
        &constantBufferDesc,
        nullptr,
        &m_constantBuffer
    )
);

// Specify the view transform corresponding to a camera position of
// X = 0, Y = 1, Z = 2. For a generalized camera class, see Lesson 5.

m_constantBufferData.view = DirectX::XMFLOAT4X4(
    -1.00000000f, 0.00000000f, 0.00000000f, 0.00000000f,
    0.00000000f, 0.89442718f, 0.44721359f, 0.00000000f,

```

```

0.00000000f, 0.44721359f, -0.89442718f, -2.23606800f,
0.00000000f, 0.00000000f, 0.00000000f, 1.00000000f
);
});

```

3. Creating textures and samplers

Here, we apply texture data to a cube rather than applying colors as in the previous tutorial, [Using depth and effects on primitives](#).

We use raw texture data to create textures.

To create textures and samplers

1. First, we read raw texture data from the `texturedata.bin` file on disk.
2. Next, we construct a `D3D11_SUBRESOURCE_DATA` structure that references that raw texture data.
3. Then, we populate a `D3D11_TEXTURE2D_DESC` structure to describe the texture. We then pass the `D3D11_SUBRESOURCE_DATA` and `D3D11_TEXTURE2D_DESC` structures in a call to `ID3D11Device::CreateTexture2D` to create the texture.
4. Next, we create a shader-resource view of the texture so shaders can use the texture. To create the shader-resource view, we populate a `D3D11_SHADER_RESOURCE_VIEW_DESC` to describe the shader-resource view and pass the shader-resource view description and the texture to `ID3D11Device::CreateShaderResourceView`. In general, you match the view description with the texture description.
5. Next, we create sampler state for the texture. This sampler state uses the relevant texture data to define how the color for a particular texture coordinate is determined. We populate a `D3D11_SAMPLER_DESC` structure to describe the sampler state. We then pass the `D3D11_SAMPLER_DESC` structure in a call to `ID3D11Device::CreateSamplerState` to create the sampler state.
6. Finally, we declare a *degree* variable that we will use to animate the cube by rotating it every frame.

C++/CX

```

// Load the raw texture data from disk and construct a subresource
// description that references it.
auto loadTDTask = DX::ReadDataAsync(L"texturedata.bin");

auto constructSubresourceTask = loadTDTask.then([this](const
std::vector<byte>& textureData)
{
    D3D11_SUBRESOURCE_DATA textureSubresourceData = { 0 };
    textureSubresourceData.pSysMem = textureData.data();

```

```

    // Specify the size of a row in bytes, known as a priori about the
    texture data.
    textureSubresourceData.SysMemPitch = 1024;

    // As this is not a texture array or 3D texture, this parameter is
    ignored.
    textureSubresourceData.SysMemSlicePitch = 0;

    // Create a texture description from information known as a priori about
    the data.
    // Generalized texture loading code can be found in the Resource Loading
    sample.
    D3D11_TEXTURE2D_DESC textureDesc = { 0 };
    textureDesc.Width = 256;
    textureDesc.Height = 256;
    textureDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    textureDesc.Usage = D3D11_USAGE_DEFAULT;
    textureDesc.CPUAccessFlags = 0;
    textureDesc.MiscFlags = 0;

    // Most textures contain more than one MIP level. For simplicity, this
    sample uses only one.
    textureDesc.MipLevels = 1;

    // As this will not be a texture array, this parameter is ignored.
    textureDesc.ArraySize = 1;

    // Don't use multi-sampling.
    textureDesc.SampleDesc.Count = 1;
    textureDesc.SampleDesc.Quality = 0;

    // Allow the texture to be bound as a shader resource.
    textureDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

    ComPtr<ID3D11Texture2D> texture;
    DX::ThrowIfFailed(
        m_d3dDevice->CreateTexture2D(
            &textureDesc,
            &textureSubresourceData,
            &texture
        )
    );

    // Once the texture is created, we must create a shader resource view of
    it
    // so that shaders may use it. In general, the view description will
    match
    // the texture description.
    D3D11_SHADER_RESOURCE_VIEW_DESC textureViewDesc;
    ZeroMemory(&textureViewDesc, sizeof(textureViewDesc));
    textureViewDesc.Format = textureDesc.Format;
    textureViewDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
    textureViewDesc.Texture2D.MipLevels = textureDesc.MipLevels;
    textureViewDesc.Texture2D.MostDetailedMip = 0;

```

```

ComPtr<ID3D11ShaderResourceView> textureView;
DX::ThrowIfFailed(
    m_d3dDevice->CreateShaderResourceView(
        texture.Get(),
        &textureViewDesc,
        &textureView
    )
);

// Once the texture view is created, create a sampler. This defines how
the color
// for a particular texture coordinate is determined using the relevant
texture data.
D3D11_SAMPLER_DESC samplerDesc;
ZeroMemory(&samplerDesc, sizeof(samplerDesc));

samplerDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_LINEAR;

// The sampler does not use anisotropic filtering, so this parameter is
ignored.
samplerDesc.MaxAnisotropy = 0;

// Specify how texture coordinates outside of the range 0..1 are
resolved.
samplerDesc.AddressU = D3D11_TEXTURE_ADDRESS_WRAP;
samplerDesc.AddressV = D3D11_TEXTURE_ADDRESS_WRAP;
samplerDesc.AddressW = D3D11_TEXTURE_ADDRESS_WRAP;

// Use no special MIP clamping or bias.
samplerDesc.MipLODBias = 0.0f;
samplerDesc.MinLOD = 0;
samplerDesc.MaxLOD = D3D11_FLOAT32_MAX;

// Don't use a comparison function.
samplerDesc.ComparisonFunc = D3D11_COMPARISON_NEVER;

// Border address mode is not used, so this parameter is ignored.
samplerDesc.BorderColor[0] = 0.0f;
samplerDesc.BorderColor[1] = 0.0f;
samplerDesc.BorderColor[2] = 0.0f;
samplerDesc.BorderColor[3] = 0.0f;

ComPtr<ID3D11SamplerState> sampler;
DX::ThrowIfFailed(
    m_d3dDevice->CreateSamplerState(
        &samplerDesc,
        &sampler
    )
);
});

// This value will be used to animate the cube by rotating it every frame;
float degree = 0.0f;

```

4. Rotating and drawing the textured cube and presenting the rendered image

As in the previous tutorials, we enter an endless loop to continually render and display the scene. We call the `rotationY` inline function (`BasicMath.h`) with a rotation amount to set values that will rotate the cube's model matrix around the Y axis. We then call `ID3D11DeviceContext::UpdateSubresource` to update the constant buffer and rotate the cube model. Next, we call `ID3D11DeviceContext::OMSetRenderTargets` to specify the render target and the depth-stencil view. We call `ID3D11DeviceContext::ClearRenderTargetView` to clear the render target to a solid blue color and call `ID3D11DeviceContext::ClearDepthStencilView` to clear the depth buffer.

In the endless loop, we also draw the textured cube on the blue surface.

To draw the textured cube

1. First, we call `ID3D11DeviceContext::IASetInputLayout` to describe how vertex buffer data is streamed into the input-assembler stage.
2. Next, we call `ID3D11DeviceContext::IASetVertexBuffers` and `ID3D11DeviceContext::IASetIndexBuffer` to bind the vertex and index buffers to the input-assembler stage.
3. Next, we call `ID3D11DeviceContext::IASetPrimitiveTopology` with the `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP` value to specify for the input-assembler stage to interpret the vertex data as a triangle strip.
4. Next, we call `ID3D11DeviceContext::VSSetShader` to initialize the vertex shader stage with the vertex shader code and `ID3D11DeviceContext::PSSetShader` to initialize the pixel shader stage with the pixel shader code.
5. Next, we call `ID3D11DeviceContext::VSSetConstantBuffers` to set the constant buffer that is used by the vertex shader pipeline stage.
6. Next, we call `PSSetShaderResources` to bind the shader-resource view of the texture to the pixel shader pipeline stage.
7. Next, we call `PSSetSamplers` to set the sampler state to the pixel shader pipeline stage.
8. Finally, we call `ID3D11DeviceContext::DrawIndexed` to draw the cube and submit it to the rendering pipeline.

As in the previous tutorials, we call `IDXGISwapChain::Present` to present the rendered image to the window.

C++/CX

```
// Update the constant buffer to rotate the cube model.  
m_constantBufferData.model = DirectX::XMMatrixRotationY(-degree);
```

```

degree += 1.0f;

m_d3dDeviceContext->UpdateSubresource(
    m_constantBuffer.Get(),
    0,
    nullptr,
    &m_constantBufferData,
    0,
    0
);

// Specify the render target and depth stencil we created as the output
// target.
m_d3dDeviceContext->OMSetRenderTargets(
    1,
    m_renderTargetView.GetAddressOf(),
    m_depthStencilView.Get()
);

// Clear the render target to a solid color, and reset the depth stencil.
const float clearColor[4] = { 0.071f, 0.04f, 0.561f, 1.0f };
m_d3dDeviceContext->ClearRenderTargetView(
    m_renderTargetView.Get(),
    clearColor
);

m_d3dDeviceContext->ClearDepthStencilView(
    m_depthStencilView.Get(),
    D3D11_CLEAR_DEPTH,
    1.0f,
    0
);

m_d3dDeviceContext->IASetInputLayout(inputLayout.Get());

// Set the vertex and index buffers, and specify the way they define
// geometry.
UINT stride = sizeof(BasicVertex);
UINT offset = 0;
m_d3dDeviceContext->IASetVertexBuffers(
    0,
    1,
    vertexBuffer.GetAddressOf(),
    &stride,
    &offset
);

m_d3dDeviceContext->IASetIndexBuffer(
    indexBuffer.Get(),
    DXGI_FORMAT_R16_UINT,
    0
);

m_d3dDeviceContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);

```

```

// Set the vertex and pixel shader stage state.
m_d3dDeviceContext->VSSetShader(
    vertexShader.Get(),
    nullptr,
    0
);

m_d3dDeviceContext->VSSetConstantBuffers(
    0,
    1,
    m_constantBuffer.GetAddressOf()
);

m_d3dDeviceContext->PSSetShader(
    pixelShader.Get(),
    nullptr,
    0
);

m_d3dDeviceContext->PSSetShaderResources(
    0,
    1,
    textureView.GetAddressOf()
);

m_d3dDeviceContext->PSSetSamplers(
    0,
    1,
    sampler.GetAddressOf()
);

// Draw the cube.
m_d3dDeviceContext->DrawIndexed(
    ARRAYSIZE(cubeIndices),
    0,
    0
);

// Present the rendered image to the window. Because the maximum frame
// latency is set to 1,
// the render loop will generally be throttled to the screen refresh rate,
// typically around
// 60 Hz, by sleeping the application on Present until the screen is
// refreshed.
DX::ThrowIfFailed(
    m_swapChain->Present(1, 0)
);

```

Summary

In this topic, we loaded raw texture data, and applied that data to a 3D primitive.

Create and display a basic mesh

Article • 10/20/2022

3-D Universal Windows Platform (UWP) games typically use polygons to represent objects and surfaces in the game. The lists of vertices that comprise the structure of these polygonal objects and surfaces are called meshes. Here, we create a basic mesh for a cube object and provide it to the shader pipeline for rendering and display.

Important The example code included here uses types (such as `DirectX::XMFLOAT3` and `DirectX::XMFLOAT4X4`) and inline methods declared in `DirectXMath.h`. If you're cutting and pasting this code, `#include <DirectXMath.h>` in your project.

What you need to know

Technologies

- [Direct3D](#)

Prerequisites

- Basic knowledge of linear algebra and 3-D coordinate systems
- A Visual Studio 2015 or later Direct3D template

Instructions

These steps will show you how to create a basic mesh cube.

Step 1: Construct the mesh for the model

In most games, the mesh for a game object is loaded from a file that contains the specific vertex data. The ordering of these vertices is app-dependent, but they are usually serialized as strips or fans. Vertex data can come from any software source, or it can be created manually. It's up to your game to interpret the data in a way that the vertex shader can effectively process it.

In our example, we use a simple mesh for a cube. The cube, like any object mesh at this stage in the pipeline, is represented using its own coordinate system. The vertex shader

takes its coordinates and, by applying the transformation matrices you provide, returns the final 2-D view projection in a homogeneous coordinate system.

Define the mesh for a cube. (Or load it from a file. It's your call!)

C++

```
SimpleCubeVertex cubeVertices[] =
{
    { DirectX::XMFLOAT3(-0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(0.0f, 1.0f, 0.0f) }, // +Y (top face)
    { DirectX::XMFLOAT3( 0.5f, 0.5f, -0.5f), DirectX::XMFLOAT3(1.0f, 1.0f, 0.0f) },
    { DirectX::XMFLOAT3( 0.5f, 0.5f,  0.5f), DirectX::XMFLOAT3(1.0f, 1.0f, 1.0f) },
    { DirectX::XMFLOAT3(-0.5f, 0.5f,  0.5f), DirectX::XMFLOAT3(0.0f, 1.0f, 1.0f) },

    { DirectX::XMFLOAT3(-0.5f, -0.5f,  0.5f), DirectX::XMFLOAT3(0.0f, 0.0f, 1.0f) }, // -Y (bottom face)
    { DirectX::XMFLOAT3( 0.5f, -0.5f,  0.5f), DirectX::XMFLOAT3(1.0f, 0.0f, 1.0f) },
    { DirectX::XMFLOAT3( 0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(1.0f, 0.0f, 0.0f) },
    { DirectX::XMFLOAT3(-0.5f, -0.5f, -0.5f), DirectX::XMFLOAT3(0.0f, 0.0f, 0.0f) },
};
```

The cube's coordinate system places the center of the cube at the origin, with the y-axis running top to bottom using a left-handed coordinate system. Coordinate values are expressed as 32-bit floating values between -1 and 1.

In each bracketed pairing, the second DirectX::XMFLOAT3 value group specifies the color associated with the vertex as an RGB value. For example, the first vertex at (-0.5, 0.5, -0.5) has a full green color (the G value is set to 1.0, and the "R" and "B" values are set to 0).

Therefore, you have 8 vertices, each with a specific color. Each vertex/color pairing is the complete data for a vertex in our example. When you specify our vertex buffer, you must keep this specific layout in mind. We provide this input layout to the vertex shader so it can understand your vertex data.

Step 2: Set up the input layout

Now, you have the vertices in memory. But, your graphics device has its own memory, and you use Direct3D to access it. To get your vertex data into the graphics device for processing, you need to clear the way, as it were: you must declare how the vertex data

is laid out so that the graphics device can interpret it when it gets it from your game. To do that, you use [ID3D11InputLayout](#).

Declare and set the input layout for the vertex buffer.

C++

```
const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "COLOR", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
};

ComPtr<ID3D11InputLayout> inputLayout;
m_d3dDevice->CreateInputLayout(
    basicVertexLayoutDesc,
    ARRAYSIZE(basicVertexLayoutDesc),
    vertexShaderBytecode->Data,
    vertexShaderBytecode->Length,
    &inputLayout)
);
```

In this code, you specify a layout for the vertices, specifically, what data each element in the vertex list contains. Here, in **basicVertexLayoutDesc**, you specify two data components:

- **POSITION:** This is an HLSL semantic for position data provided to a shader. In this code, it's a DirectX::XMFLOAT3, or more specifically, a structure with 3 32-bit floating point values that correspond to a 3D coordinate (x, y, z). You could also use a float4 if you are supplying the homogeneous "w" coordinate, and in that case, you specify DXGI_FORMAT_R32G32B32A32_FLOAT. Whether you use a DirectX::XMFLOAT3 or a float4 is up to the specific needs of your game. Just make sure that the vertex data for your mesh corresponds correctly to the format you use!

Each coordinate value is expressed as a floating point value between -1 and 1, in the object's coordinate space. When the vertex shader completes, the transformed vertex is in the homogeneous (perspective corrected) view projection space.

"But the enumeration value indicates RGB, not XYZ!" you smartly note. Good eye! In both the cases of color data and coordinate data, you typically use 3 or 4 component values, so why not use the same format for both? The HLSL semantic, not the format name, indicates how the shader treats the data.

- **COLOR**: This is an HLSL semantic for color data. Like **POSITION**, it consists of 3 32-bit floating point values (DirectX::XMFLOAT3). Each value contains a color component: red (r), blue (b), or green (g), expressed as a floating number between 0 and 1.

COLOR values are typically returned as a 4-component RGBA value at the end of the shader pipeline. For this example, you will be setting the "A" alpha value to 1.0 (maximum opacity) in the shader pipeline for all pixels.

For a complete list of formats, see [DXGI_FORMAT](#). For a complete list of HLSL semantics, see [Semantics](#).

Call [ID3D11Device::CreateInputLayout](#) and create the input layout on the Direct3D device. Now, you need to create a buffer that can actually hold the data!

Step 3: Populate the vertex buffers

Vertex buffers contain the list of vertices for each triangle in the mesh. Every vertex must be unique in this list. In our example, you have 8 vertices for the cube. The vertex shader runs on the graphics device and reads from the vertex buffer, and it interprets the data based on the input layout you specified in the previous step.

In the next example, you provide a description and a subresource for the buffer, which tell Direct3D a number of things about the physical mapping of the vertex data and how to treat it in memory on the graphics device. This is necessary because you use a generic [ID3D11Buffer](#), which could contain anything! The [D3D11_BUFFER_DESC](#) and [D3D11_SUBRESOURCE_DATA](#) structures are supplied to ensure that Direct3D understands the physical memory layout of the buffer, including the size of each vertex element in the buffer as well as the maximum size of the vertex list. You can also control access to the buffer memory here and how it is traversed, but that's a bit beyond the scope of this tutorial.

After you configure the buffer, you call [ID3D11Device::CreateBuffer](#) to actually create it. Obviously, if you have more than one object, create buffers for each unique model.

Declare and create the vertex buffer.

C++

```
D3D11_BUFFER_DESC vertexBufferDesc = {0};
vertexBufferDesc.ByteWidth = sizeof(SimpleCubeVertex) *
    ARRAYSIZE(cubeVertices);
vertexBufferDesc.Usage = D3D11_USAGE_DEFAULT;
vertexBufferDesc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
vertexBufferDesc.CPUAccessFlags = 0;
```

```

vertexBufferDesc.MiscFlags = 0;
vertexBufferDesc.StructureByteStride = 0;

D3D11_SUBRESOURCE_DATA vertexBufferData;
vertexBufferData.pSysMem = cubeVertices;
vertexBufferData.SysMemPitch = 0;
vertexBufferData.SysMemSlicePitch = 0;

ComPtr<ID3D11Buffer> vertexBuffer;
m_d3dDevice->CreateBuffer(
    &vertexBufferDesc,
    &vertexBufferData,
    &vertexBuffer);

```

Vertices loaded. But what's the order of processing these vertices? That's handled when you provide a list of indices to the vertices—the ordering of these indices is the order in which the vertex shader processes them.

Step 4: Populate the index buffers

Now, you provide a list of the indices for each of the vertices. These indices correspond to the position of the vertex in the vertex buffer, starting with 0. To help you visualize this, consider that each unique vertex in your mesh has a unique number assigned to it, like an ID. This ID is the integer position of the vertex in the vertex buffer.

